

# Matlab Praktikum Maschinelles Sehen

Prof. Dr.-Ing. Volker Willert



Wintersemester  
Übungsblatt 3

In dieser Praxiseinheit werden folgende Punkte behandelt:

- Zerlegung einer Projektionsmatrix mit einer RQ-Zerlegung
- Berechnung einer Homographie über die Singulärwertzerlegung
- Kalibrieren einer Webcam mit der cameraCalibrator App
- Lokalisierung & Tracking einer Kamera mit einer visuellen Landmarke

Benutzen Sie auch die Matlab-Hilfe `help` um die Aufgaben zu lösen.

---

## Aufgabe 3.1: Zerlegung einer Projektionsmatrix mit einer RQ-Zerlegung

---

Es ist folgende Projektionsmatrix gegeben:

$$\mathbf{\Pi} = \begin{bmatrix} 10^3\sqrt{2} & 10^3 - 300\sqrt{2} & -10^3 - 300\sqrt{2} & 4 \cdot 10^4 \\ 10^3\sqrt{2} & -10^3 - 400\sqrt{2} & 10^3 - 400\sqrt{2} & 6 \cdot 10^4 \\ 0 & -0.5\sqrt{2} & -0.5\sqrt{2} & 100 \end{bmatrix} = [\mathbf{p}_1 \quad \mathbf{p}_2 \quad \mathbf{p}_3 \quad \mathbf{p}_4] = \mathbf{K} [\mathbf{R}_{rot} | \mathbf{T}] = [\mathbf{K}\mathbf{R}_{rot} | \mathbf{K}\mathbf{T}]$$

Zerlegen Sie die ersten drei Spalten der Projektionsmatrix  $[\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3]$  in die Kalibriermatrix  $\mathbf{K}$  und die Rotationsmatrix  $\mathbf{R}_{rot}$  mit Hilfe der QR-Zerlegung, die in der Matlab-Funktion `qr` implementiert ist. Berechnen Sie danach den Translationsvektor  $\mathbf{T}$  über die letzte Spalte der Projektionsmatrix  $\mathbf{p}_4$  und die inverse der Kalibriermatrix  $\mathbf{K}^{-1}$ . Gehen Sie wie folgt vor:

- a) Um eine RQ-Zerlegung einer  $3 \times 3$  Matrix  $\mathbf{M} = \mathbf{K}\mathbf{R}_{rot}$  mit einer QR-Zerlegung zu realisieren, müssen Sie die Matrix zuerst um  $90^\circ$  im Uhrzeigersinn drehen. Das geschieht mit folgender Transformation:  $\bar{\mathbf{M}} = \mathbf{M}^T \mathbf{\Gamma}$ , wobei die Transformationsmatrix folgendermaßen gegeben ist:

$$\mathbf{\Gamma} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \text{ mit } \mathbf{\Gamma} = \mathbf{\Gamma}^T = \mathbf{\Gamma}^{-1}.$$

- b) Zerlegen Sie nun die transformierte Matrix  $\bar{\mathbf{M}} = \overline{\mathbf{Q}\mathbf{R}}$  mit der Matlab-Funktion `qr` in ein Produkt aus einer orthogonalen Matrix  $\bar{\mathbf{Q}}$  und einer oberen Dreiecksmatrix  $\bar{\mathbf{R}}$ .
- c) Jetzt können Sie folgende Rücktransformationen verwenden, um die Kalibrier- und die Rotationsmatrix zu bestimmen:

$$\mathbf{K} = \mathbf{R} = \mathbf{\Gamma} \bar{\mathbf{R}}^T \mathbf{\Gamma},$$
$$\mathbf{R}_{rot} = \mathbf{Q} = \mathbf{\Gamma} \bar{\mathbf{Q}}^T.$$

- d) Überprüfen Sie die Eigenschaften der Kalibriermatrix: Alle Diagonalelemente müssen positiv sein. Um das sicherzustellen, bestimmen Sie die diagonale Vorzeichenmatrix  $\mathbf{D} = \text{diag}(\text{sign}(K_{11}), \text{sign}(K_{22}), \text{sign}(K_{33}))$ . Da die Bedingung  $\mathbf{K}\mathbf{R}_{rot} = \mathbf{K}\mathbf{D}\mathbf{D}^{-1}\mathbf{R}_{rot}$  immer erfüllt ist, erhalten Sie die Kalibriermatrix und die Rotationsmatrix über folgende Transformation:

$$\mathbf{K} = \mathbf{K}\mathbf{D},$$

$$\mathbf{R}_{rot} = \mathbf{D}^{-1}\mathbf{R}_{rot} = \mathbf{D}\mathbf{R}_{rot}.$$

- e) Schreiben Sie ihre eigene Funktion `myInvK(K)`, welche die Inverse der Kalibriermatrix berechnet, ohne die Matlab-Funktion `inv` zu verwenden.
- f) Berechnen Sie den Translationsvektor  $\mathbf{T} = \mathbf{K}^{-1}\mathbf{p}_4$ .
- g) Berechnen Sie den Ursprung der Kamera in Weltkoordinaten, indem Sie den Nullraum der Projektionsmatrix über eine Singulärwertzerlegung berechnen:

$$\mathbf{0} = \mathbf{\Pi}\mathbf{o}_W = \mathbf{K}\mathbf{R}\mathbf{o}_W + \mathbf{K}\mathbf{T}.$$

Der Nullraum entspricht dem rechtsseitigen Singulärvektor  $\mathbf{v}_{min}$  mit dem kleinsten Singulärwert ( $s_{min} = 0$ ). Überprüfen Sie ihr Ergebnis über den Zusammenhang:

$$\mathbf{o}_W = -[\mathbf{K}\mathbf{R}]^{-1}\mathbf{K}\mathbf{T}.$$

### Aufgabe 3.2: Berechnung der Pose einer Kamera aus vier bekannten Punkten auf einer Ebene

Die Berechnung der Pose einer Kamera aus vier bekannten Punkten auf einer Ebene erfolgt über die Berechnung einer Homographie zwischen bekannten 3D Weltpunkten und korrespondierenden gemessenen normierten Bildkoordinaten. Es sind folgende Matrizen gegeben: Die intrinsischen Parameter, vier bekannte Weltpunkte eines Schachbrettmusters in Zentimetern und die korrespondierenden gemessenen (nichtnormierten) Pixelkoordinaten:

$$\text{Kalibriermatrix : } \mathbf{K} = \begin{bmatrix} 2000 & 0 & 600 \\ 0 & 2000 & 800 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\text{Planare Weltpunkte : } \{\mathbf{X}_i\} = \begin{bmatrix} 0 & 0 & 20 & 20 \\ 0 & 20 & 0 & 20 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\text{Pixelkoordinaten : } \{\mathbf{x}_i\} = \begin{bmatrix} 400 & 600 & 683 & 929 \\ 600 & 334 & 883 & 664 \end{bmatrix}$$

- a) Berechnen Sie die normierten Bildkoordinaten über die inverse Kalibriermatrix und die Pixelkoordinaten.
- b) Stellen Sie die Messmatrix aus den vier Punkten über eine Direkte Lineare Transformation auf. Schreiben Sie dazu eine Funktion `mySkewMat`, die aus einem  $3 \times 3$  Vektor eine schiefsymmetrische Matrix erzeugt. Benutzen Sie dazu außerdem die Matlab-Funktion `kron` für das Kroneckerprodukt. Pro Punktepaar ergeben sich zwei Gleichungen für die Messmatrix.
- c) Lösen Sie das homogene Gleichungssystem der nach den Parametern der Homographie auf. Hinweis: Die einzelnen **Spalten der Homographiematrix** werden bei der DLT in einen  $9 \times 1$  Vektor umsortiert.
- d) Normieren Sie die berechnete Homographiematrix, so dass die ersten beiden Spalten den ersten beiden Spalten der Rotationsmatrix entsprechen und die dritte Spalte dem Translationsvektor und geben Sie Rotationsmatrix und Translationsvektor aus.



Abbildung 1: Zu Verfügung gestellte Kalibrierbilder zum Kalibrieren einer Webcam. Auflösung  $1920 \times 1280$  Pixel.

---

### Aufgabe 3.3: Kalibrieren einer Webcam mit der cameraCalibrator App

---

In dieser Aufgabe sollen Sie ihre eigene Webcam mit Matlab kalibrieren. Falls Sie keine eigene Kamera besitzen, dann benutzen Sie die zur Verfügung gestellten Aufnahmen.

- Drucken Sie sich das Kalibriermuster `Kalibriermuster.pdf` mit einer Seitenlänge von  $2\text{cm} \times 2\text{cm}$  auf ein weißes Blatt Papier und kleben Sie es auf einen flachen Karton. Das können Sie jetzt zur Kalibrierung ihrer Webcam benutzen.
- Starten Sie die `cameraCalibrator` App. Stellen Sie im Menüpunkt `Camera Properties` die Kameraauflösung auf die maximal mögliche ihrer Kamera und stellen Sie den Fokus manuell auf den Arbeitsabstand ein, welcher das Schachbrett in ihrem Arbeitsraum scharf abbildet. Wenn der Fokus nicht konstant gehalten wird, dann funktioniert die Kalibrierung der Kamera nicht! Folgen Sie nun den Anweisungen der App.
- Speichern Sie alle Kalibrierparameter oder die komplette Session in einer `*.mat`-Datei ab, damit Sie die Kalibrierdaten zukünftig benutzen können.

### Aufgabe 3.4: Lokalisierung & Tracking einer Kamera mit einer visuellen Landmarke

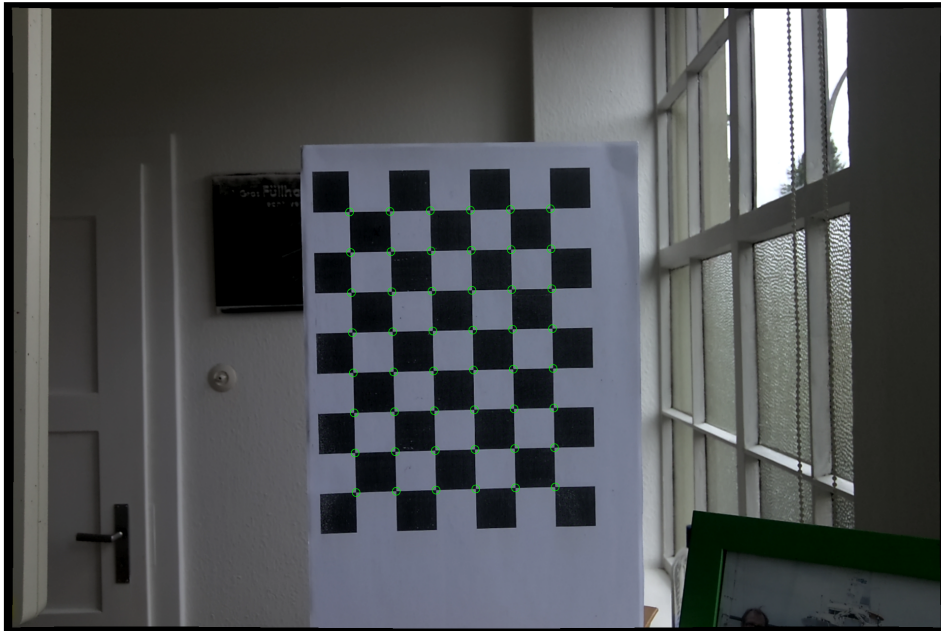


Abbildung 2: Das Schachbrett dient jetzt als visuelle Landmarke, um die Pose der Kamera in Relation zum Schachbrett zu berechnen.

Wenn Sie ihre Kamera kalibriert haben, dann können Sie jetzt das Schachbrett als bekannte visuelle Landmarke benutzen und die Pose ihrer Kamera relativ zum Schachbrett berechnen. Dazu gehen Sie wie folgt vor:

- Laden Sie ihre Kameraparameter in den Arbeitsraum von Matlab. Falls Sie keine eigene Kamera kalibriert haben, dann benutzen Sie die zur Verfügung gestellte Datei `load('cameraParams.mat')`. Laden Sie eine Aufnahme des Schachbretts in den Arbeitsraum. Falls Sie keine eigenen Aufnahmen gemacht haben, dann benutzen Sie z.B. `imread('Image64.png')`.
- Zeigen Sie das Bild an. Danach benutzen Sie die Funktion `undistortImage`, um ein verzerrungsfreies Bild zu erhalten. Zeigen Sie sich das verzerrungsfreie Bild an und betrachten Sie die Unterschiede.
- Extrahieren Sie nun die Eckpunkte des Schachbretts im Bild mit der Funktion `detectCheckerboardPoints` und generieren Sie sich die dazugehörigen Weltpunkte mit der Funktion `generateCheckerboardPoints`. Zeigen Sie sich die Punkte auf dem Bild an.
- Berechnen Sie die Pose der Kamera mit der Funktion `extrinsics` und lassen Sie sich die Pose ausgeben.
- Binden Sie den bisherigen Code in eine Schleife ein und wenden Sie den Algorithmus wiederholt auf Livebilder ihrer Webcam an, während Sie sie bewegen. Achten Sie bei der Bewegung darauf, dass das Schachbrett immer vollständig im Sichtfeld zu sehen ist. Wenn Sie möchten, können Sie sich den Verlauf der Kamerapose mit den Funktionen `extrinsicsToCameraPose` und `plotCamera` anzeigen lassen.